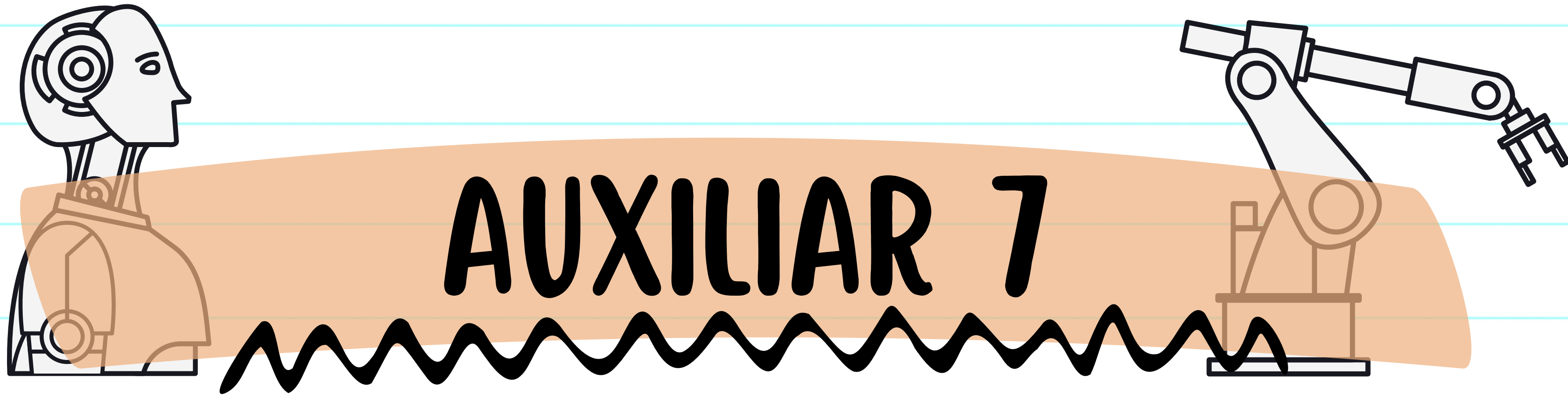




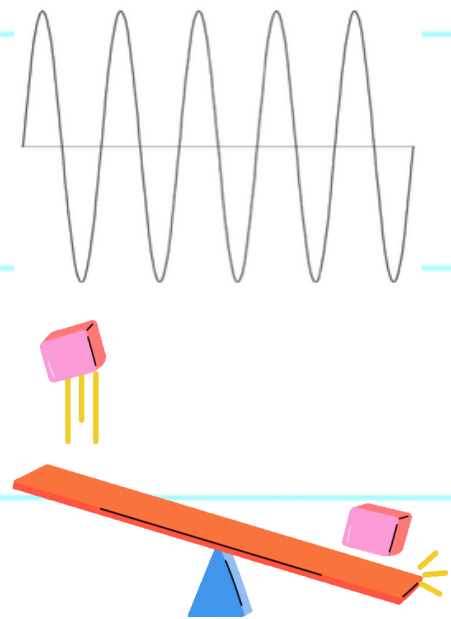
Mecatrónica - ME4250

Profesor: Harold Valenzuela

Auxiliar: Fernando Navarrete

CONTROL AUTOMÁTICO

ESTÍMULO, INFORMACIÓN



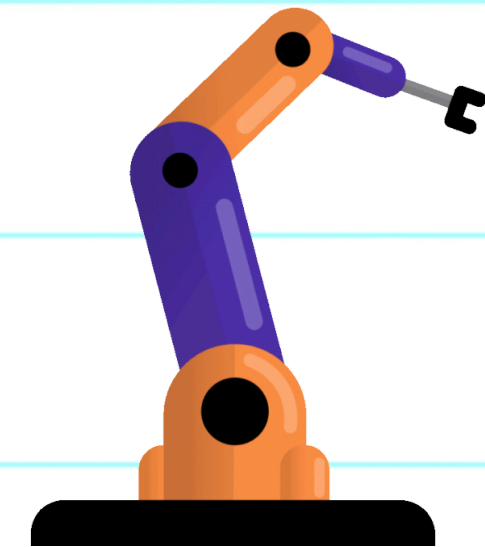
SENSOR / TRANSDUCTOR



MICRO-CONTROLADOR



ACTUACIÓN

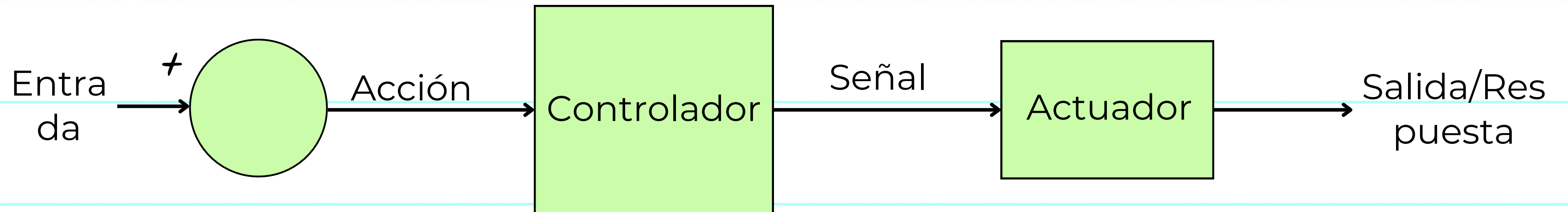


TIPOS DE CONTROL

- Control automático de lazo abierto (Open-Loop)
- Control automático con retroalimentación (Closed-Loop o Feedback control):
 - Lazo cerrado con retroalimentación negativa.
 - Lazo cerrado con retroalimentación positiva.
- Control de adelanto-retraso (Lead-Lag)
- Etc.

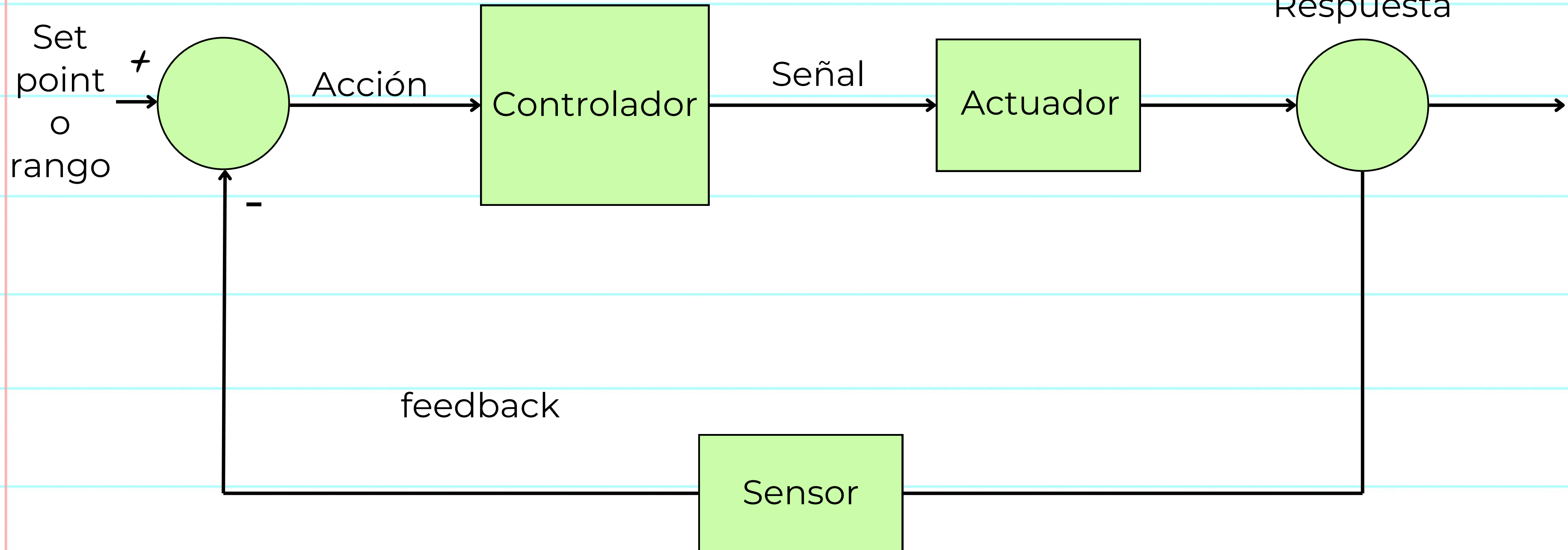
TIPOS DE CONTROL

Control automático de Lazo Abierto (Open Loop)



TIPOS DE CONTROL

Control automático de Lazo Cerrado (Closed Loop)

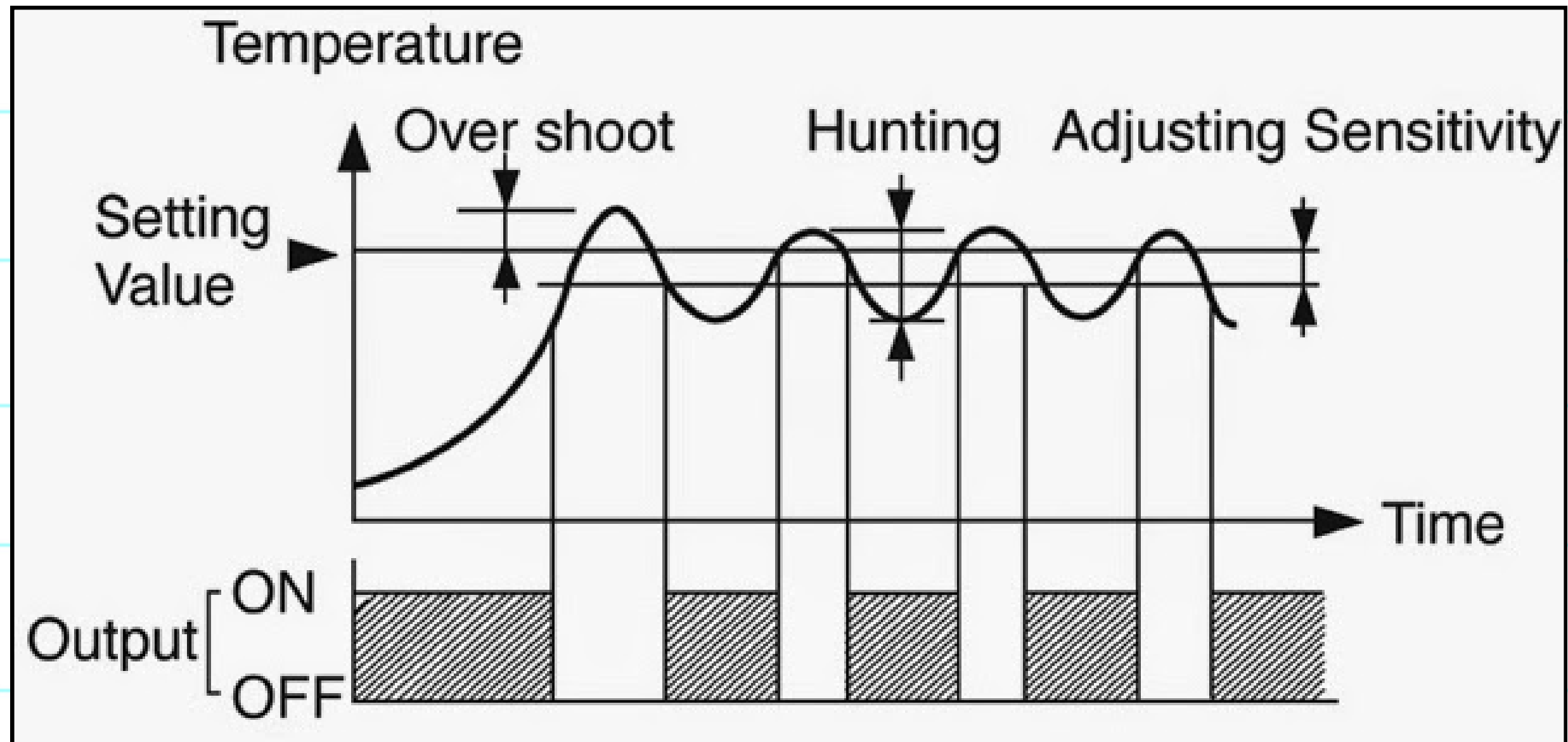


TIPOS DE CONTROLES

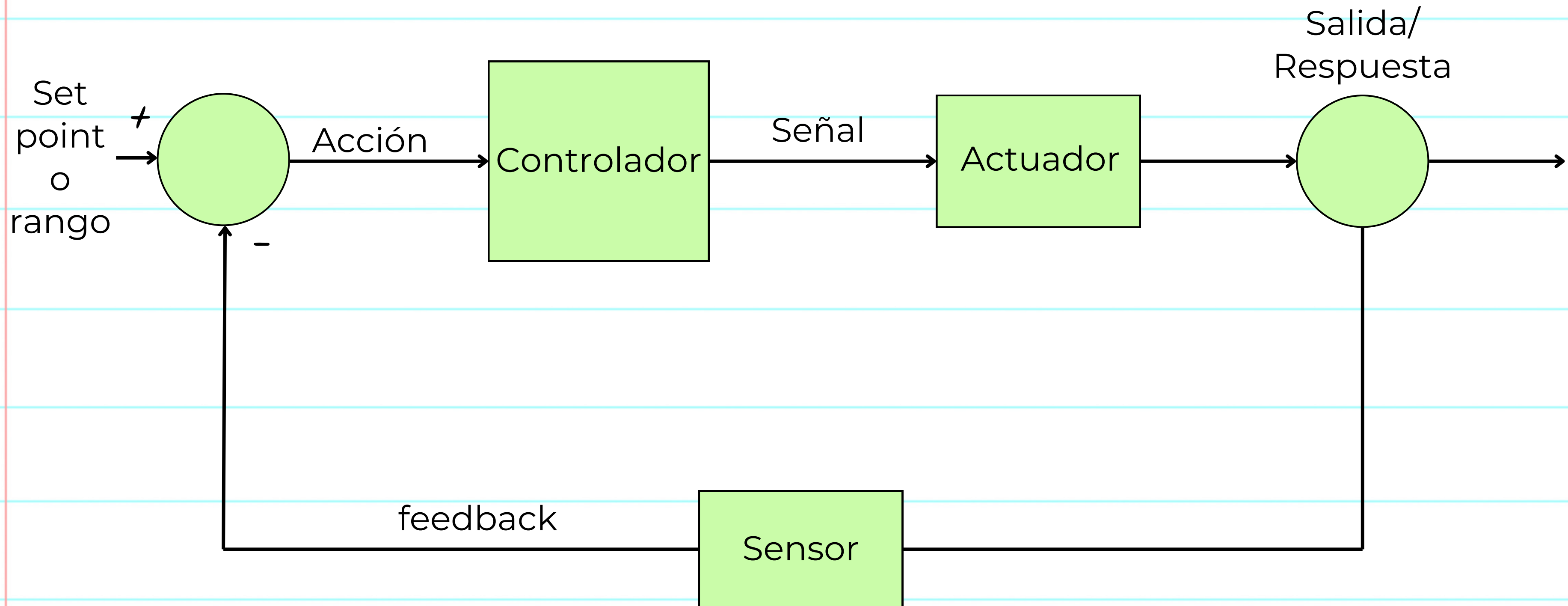
- **Control automático ON/OFF** (Todo o Nada)
- Control automático P (Proporcional)
- Control automático PI (Proporcional e integrativo)
- **Control automático PID** (Proporcional, integrativo y derivativo).
- **MPC, etc.**

TIPOS DE RESPUESTA

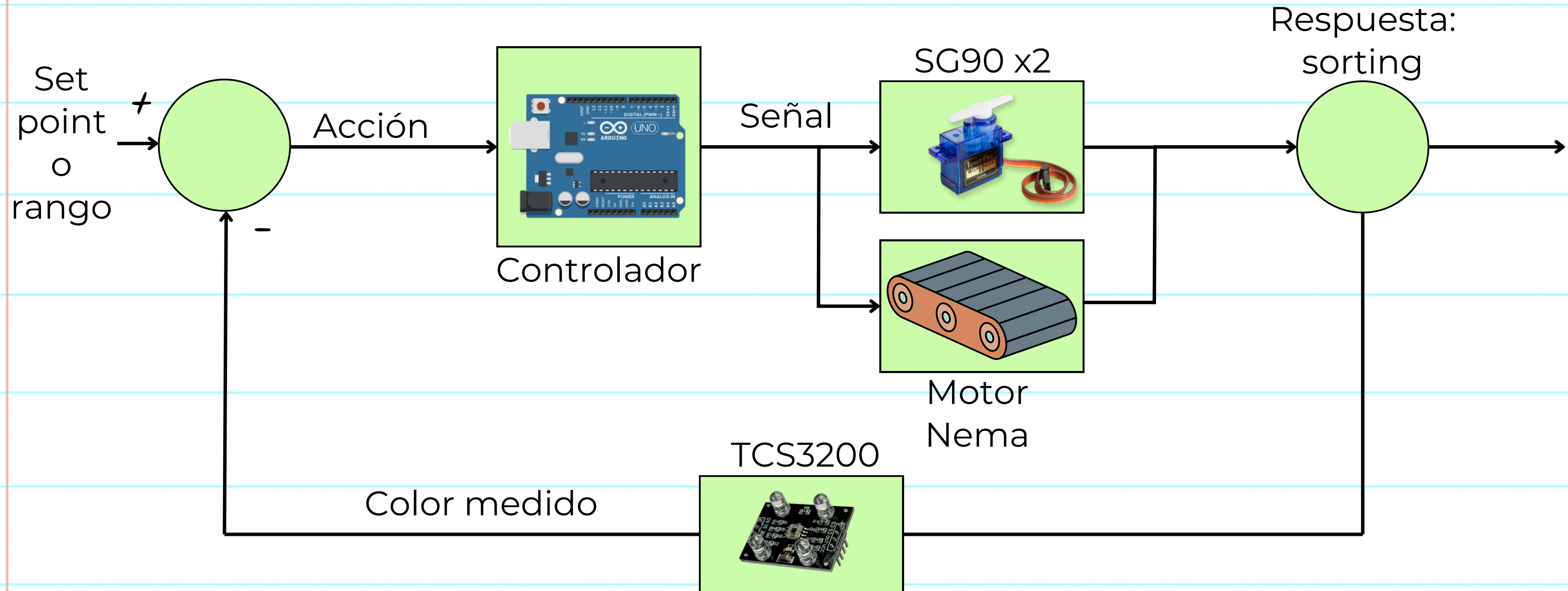
Control automático ON/OFF (Todo o Nada)



CONTROL AUTOMÁTICO: ON/OFF



CONTROL AUTOMÁTICO: DESAFÍO 2



CONTROL PID

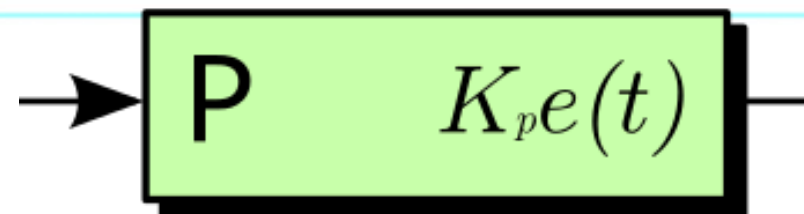
→ **P** $K_p e(t)$ — Control proporcional: “presente”

→ **I** $K_i \int_0^t e(\tau) d\tau$ — Control integrativo: “pasado”

→ **D** $K_d \frac{de(t)}{dt}$ — Control derivativo: “futuro”

Salida: $e_{total}(t) = K_p \cdot e(t) + K_I \cdot \int_0^t e(t) dt + K_D \cdot \frac{de(t)}{dt}$

CONTROL PID



Salida

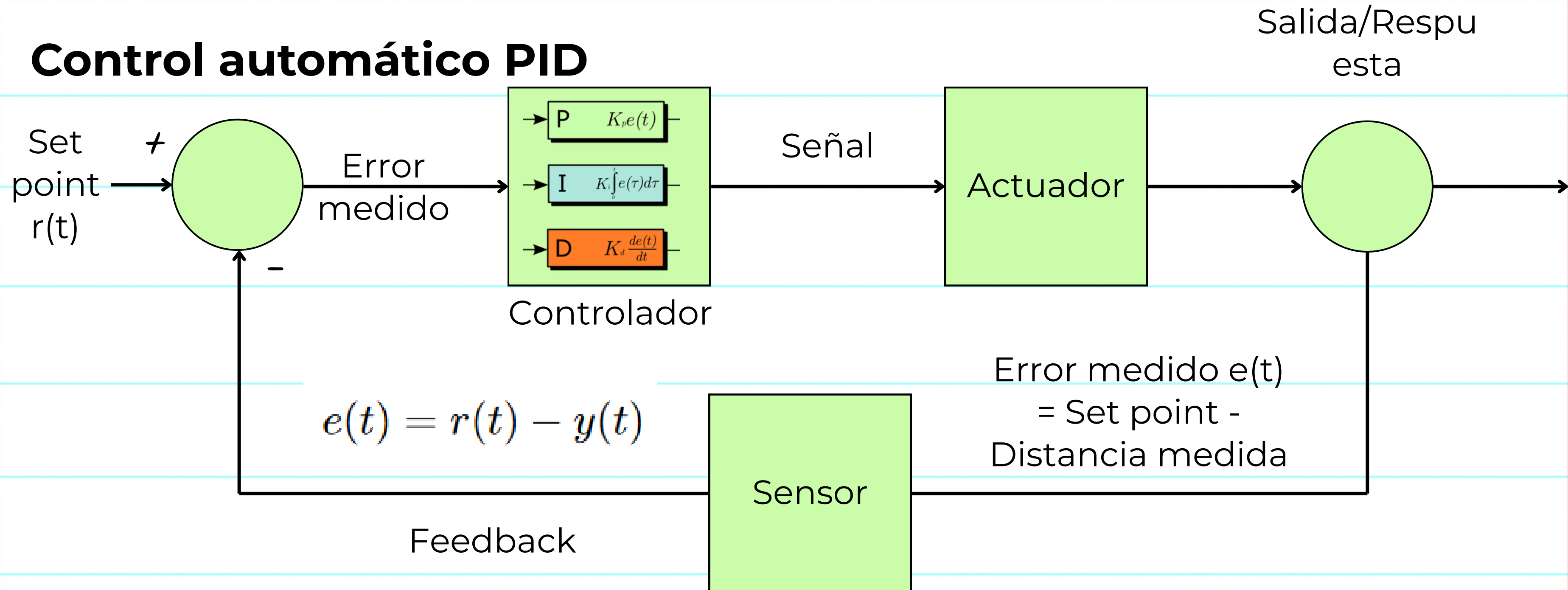
$$e_{total}(t) = K_p \cdot e(t) + K_I \cdot \int_0^t e(t) + K_D \cdot \frac{de(t)}{dt}$$

Salida discretizada:

$$e_{total}(t) = K_p \cdot e(t) + K_I \cdot \sum_0^t e(t) \cdot \Delta t + K_D \cdot \frac{e(t) - e(t-1)}{\Delta t}$$

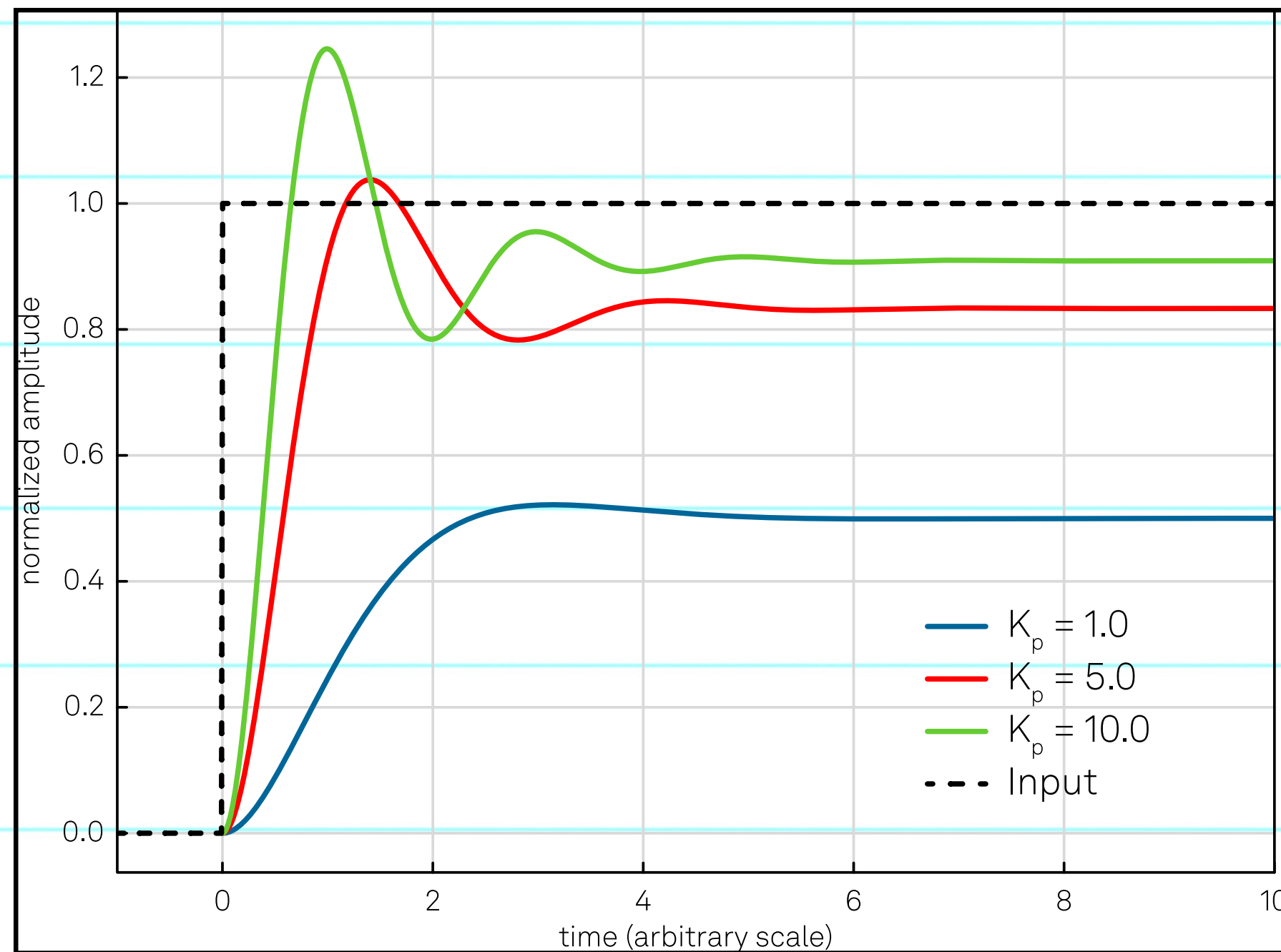
TIPOS DE RESPUESTA

Control automático PID



TIPOS DE RESPUESTA

Control automático PID



MÉTODO DE ZIEGLER-NICHOLS (PID)

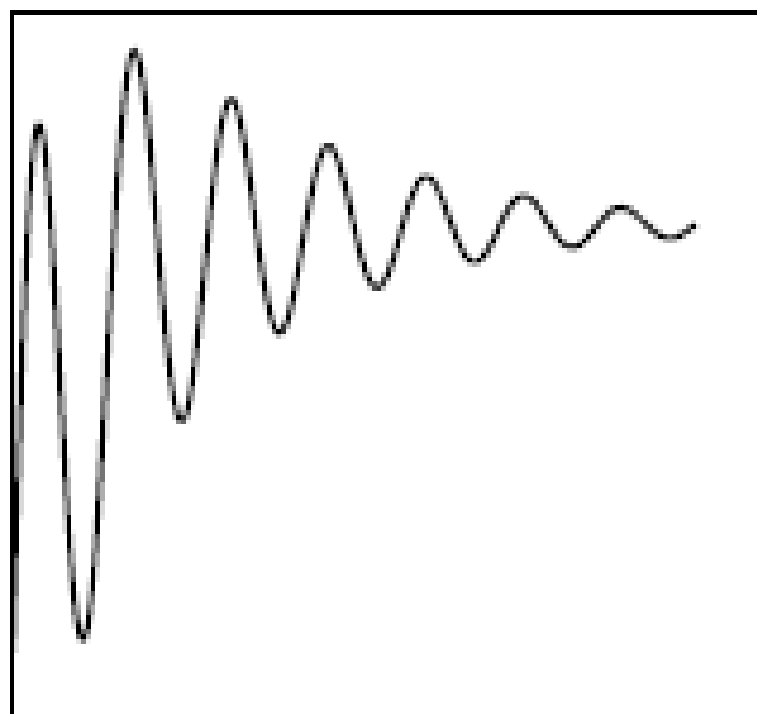
PASOS A SEGUIR:

- DEFINIR TODAS LAS GANANCIAS EN CERO
- INCREMENTAR GRADUALMENTE K_P HASTA QUE EL SISTEMA ALCANCE UNA OSCILACIÓN ESTACIONARIA, EL VALOR OBTENIDO SERÁ LA “GANANCIA ÚLTIMA”
- DETERMINAR EL PERÍODO DE OSCILACIÓN ASOCIADO A ESTE ESTADO ESTACIONARIO DADO POR LA GANANCIA ÚLTIMA.

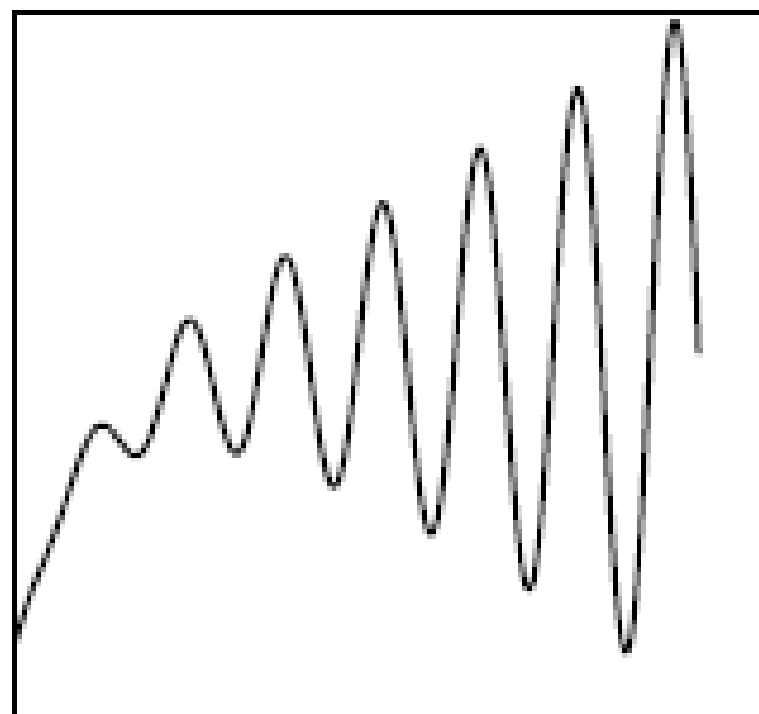
$$K_P = K_I = K_D = 0 \quad K_U := K_P^{steady} \quad T_U := T_U^{steady} [s]$$

Referencia

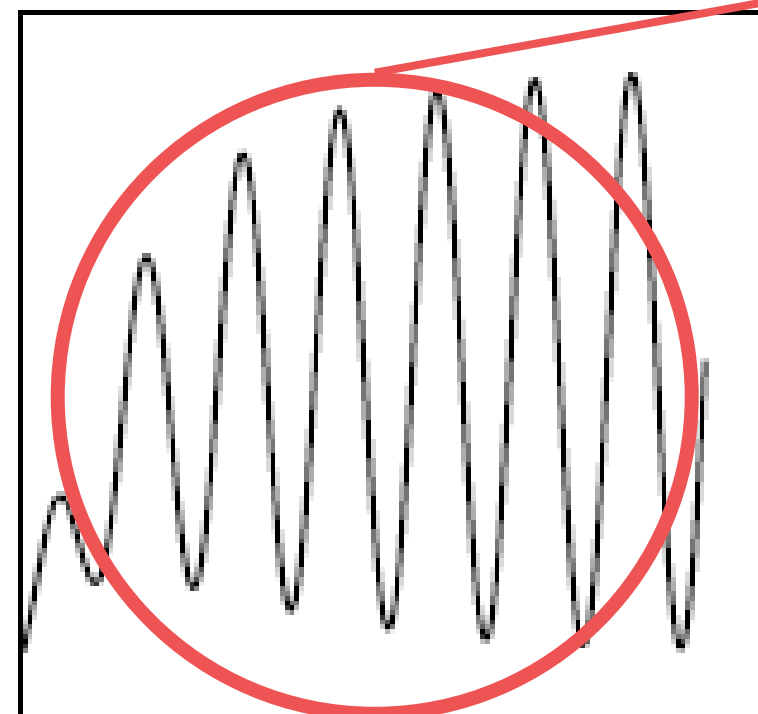
MÉTODO DE ZIEGLER-NICHOLS (PID)



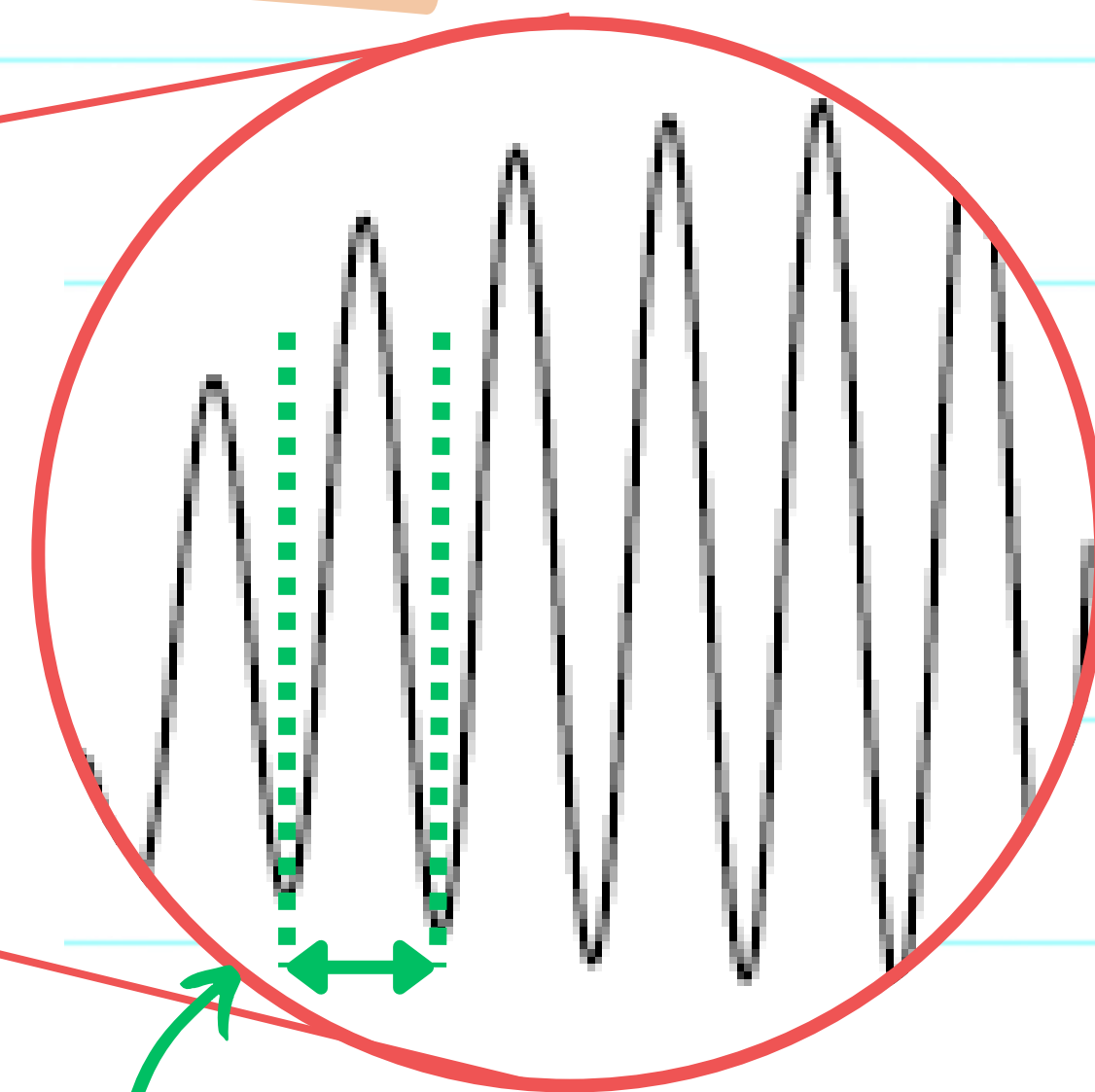
Gain < K_u



Gain > K_u



Gain = K_u



$$K_P = K_I = K_D = 0 \quad K_U := K_P^{steady}$$

$$T_U := T_U^{steady} [s]$$

MÉTODO DE ZIEGLER-NICHOLS (PID)

PASOS A SEGUIR:

- SEGÚN EL TIPO DE CONTROL REQUERIDO, CALCULAR LAS CONSTANTES SUGERIDAS POR LA TABLA MOSTRADA A CONTINUACIÓN:

Tipo de Control	K_P	T_I	T_D	K_I	K_D
P	$0.5K_u$	—	—	—	—
PI	$0.45K_u$	$0.83\bar{3}T_u$	—	$0.54K_u/T_u$	—
PD	$0.8K_u$	—	$0.125T_u$	—	$0.10K_uT_u$
PID clásico	$0.6K_u$	$0.5T_u$	$0.125T_u$	$1.2K_u/T_u$	$0.075K_uT_u$
Regla de Pessen	$0.7K_u$	$0.4T_u$	$0.15T_u$	$1.75K_u/T_u$	$0.105K_uT_u$
Overshoot	$0.33K_u$	$0.5T_u$	$0.33T_u$	$0.66K_u/T_u$	$0.11K_uT_u$
No Overshoot	$0.2K_u$	$0.5T_u$	$0.33T_u$	$0.4K_u/T_u$	$0.066K_uT_u$

MÉTODO DE ZIEGLER-NICHOLS (PID)

SUGERENCIAS:

- SE SUGIERE IR AJUSTANDO LAS CONSTANTES CALCULADAS CON EL FIN DE COMPENSAR LOS RETRASOS FÍSICOS DEL SISTEMA.
- PRUEBEN A DISMINUIR EN TOLERANCIAS DE 0.1 PARA CADA CONSTANTE, CONSIDERANDO AJUSTAR EN EL SIGUIENTE ORDEN: PROPORCIONAL, DERIVATIVO E INTEGRATIVO.
- ADECUAR SU CONTROL AUTOMÁTICO DE ACUERDO A LOS CASOS DE LA TABLA.

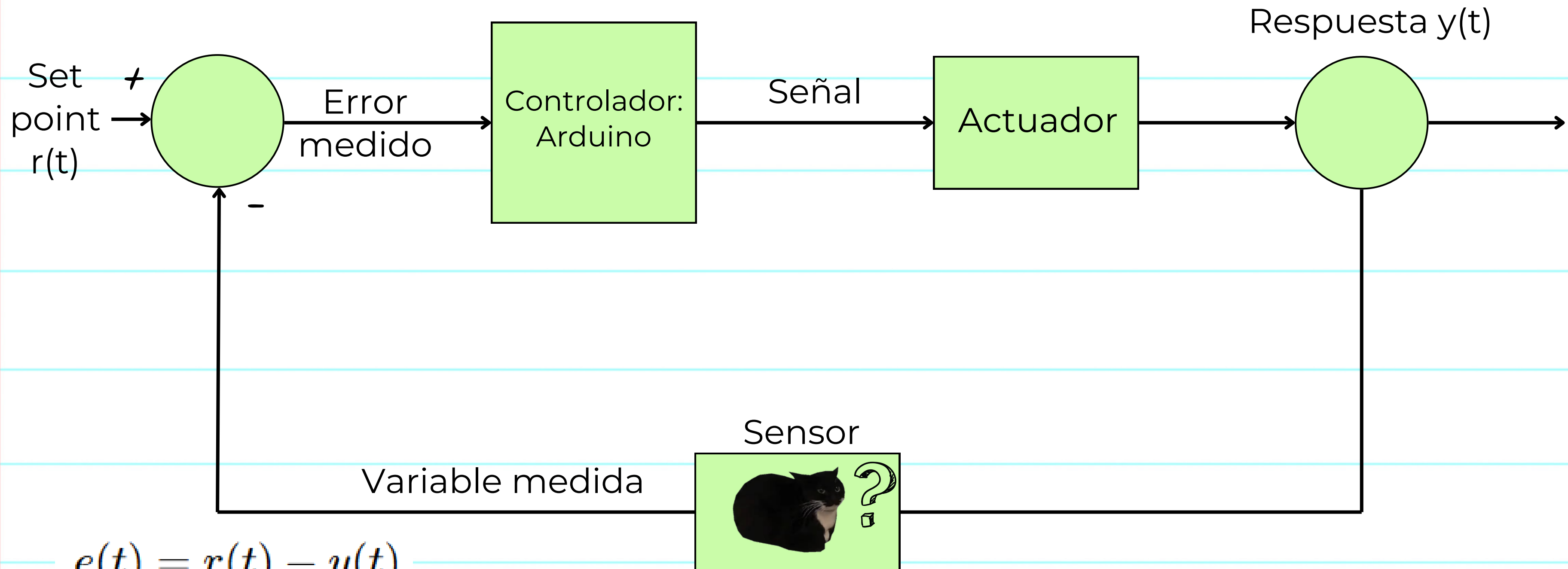
MÉTODO EMPÍRICO (A MANO)

BASICAMENTE:

CAMBIAR LAS CONSTANTES HASTA QUE FUNCIONE.

- SE PARTE CON LA KP
- LUEGO KD
- ÚLTIMO KI

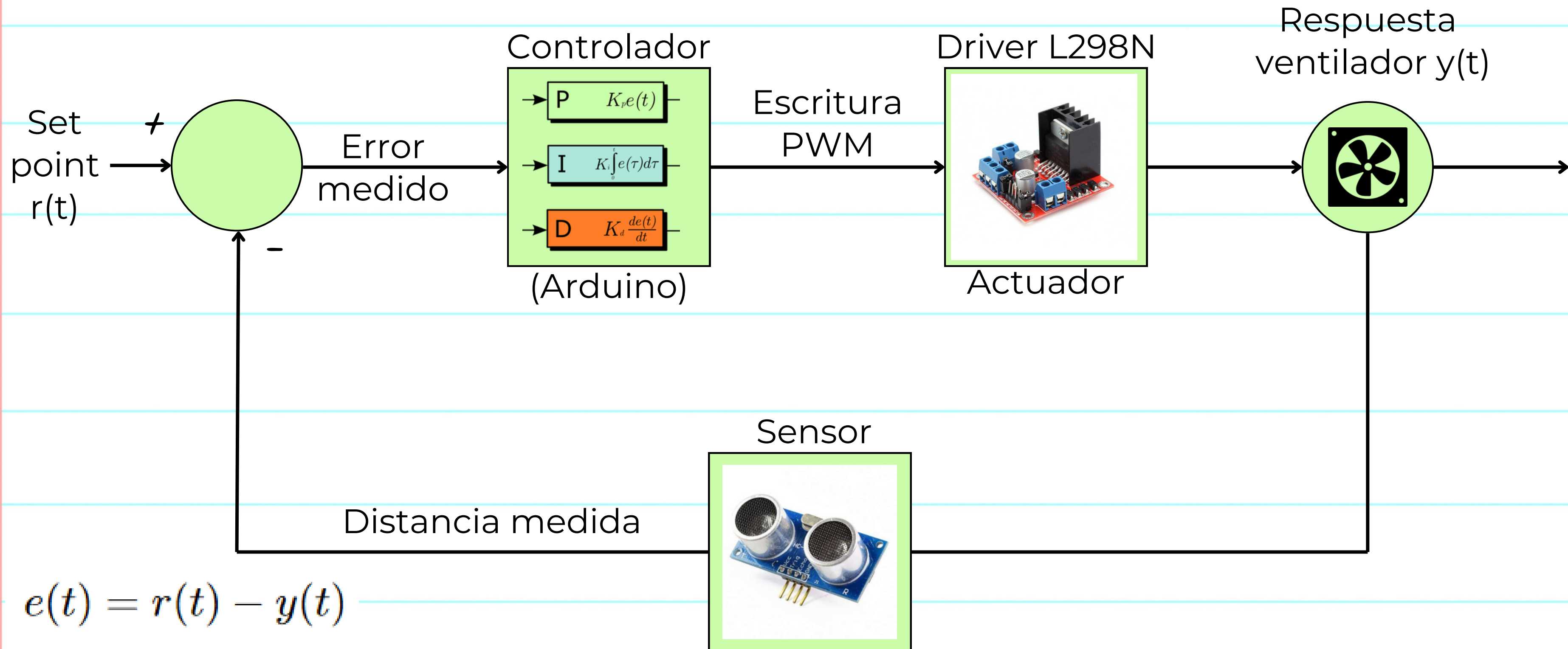
CONTROL PID APLICADO



$$e(t) = r(t) - y(t)$$

Error medido $e(t)$ = Set point - Variable medida

CONTROL PID APLICADO



Error medido $e(t)$ = Set point - Distancia medida

CONTROL PID APLICADO: CÓDIGO

// --- CONFIGURACIÓN DE PINES ---//

Sensor Ultrasonico HC-SR04

const int PIN_TRIG = 9;

const int PIN_ECHO = 10;

// Driver L298N (Control de un canal de Motor DC)

const int PIN_ENA = 5; // Pin PWM para controlar la velocidad del motor

const int PIN_IN1 = 6; // Dirección del motor pin 1

const int PIN_IN2 = 7; // Dirección del motor pin 2

// --- PARÁMETROS DEL CONTROLADOR PID ---

double Setpoint = 15.0; // Distancia objetivo deseada en centímetros (ej: 15 cm)

// Constantes de sintonización (Valores iniciales propuestos para ajuste en aula)

double Kp = 12.0; // Ganancia Proporcional

double Ki = 0.1; // Ganancia Integral

double Kd = 4.0; // Ganancia Derivativa

CONTROL PID APLICADO: CÓDIGO

// Variables internas del lazo de control

```
double error_actual = 0;  
double error_anterior = 0;  
double integral = 0;  
double derivativa = 0;  
double salida_pid = 0;
```

// Variables de tiempo discretizado

```
unsigned long tiempo_anterior = 0;  
const unsigned long dt_fijo = 50; // Intervalo de muestreo estricto en milisegundos (50 ms)
```

CONTROL PID APLICADO: CÓDIGO

// --- FUNCIÓN DE SOPORTE ---

// Lectura limpia del sensor ultrasónico utilizando la velocidad del sonido

```
double obtenerDistanciaCM() {  
    digitalWrite(PIN_TRIG, LOW);  
    delayMicroseconds(2);  
    digitalWrite(PIN_TRIG, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(PIN_TRIG, LOW);  
  
    // Medición del pulso de retorno en microsegundos  
    long duracion = pulseIn(PIN_ECHO, HIGH, 30000); // Timeout de 30ms para evitar bloqueos  
  
    // Si no hay lectura válida, retornar un valor seguro o el Setpoint para congelar la acción  
    if (duracion == 0) return Setpoint;  
  
    // Distancia = (Tiempo * Velocidad del sonido) / 2  
    return (duracion * 0.0343) / 2.0;  
}
```

CONTROL PID APLICADO: CÓDIGO

```
void setup() {  
  Serial.begin(9600);  
  
  // Configuración de Entradas/Salidas del sensor  
  pinMode(PIN_TRIG, OUTPUT);  
  pinMode(PIN_ECHO, INPUT);  
  
  // Configuración de Entradas/Salidas del L298N  
  pinMode(PIN_ENA, OUTPUT);  
  pinMode(PIN_IN1, OUTPUT);  
  pinMode(PIN_IN2, OUTPUT);  
  
  // Configurar dirección del motor  
  digitalWrite(PIN_IN1, HIGH);  
  digitalWrite(PIN_IN2, LOW);  
  tiempo_anterior = millis();  
}
```

CONTROL PID APLICADO: CÓDIGO

```
void loop() {
```

```
    unsigned long tiempo_actual = millis();
```

```
    // Ejecución temporizada estricta para garantizar la validez matemática de Ki y Kd
```

```
    if (tiempo_actual - tiempo_anterior >= dt_fijo) {
```

```
        double dt = (tiempo_actual - tiempo_anterior) / 1000.0; // Convertir milisegundos a segundos
        tiempo_anterior = tiempo_actual;
```

```
        // 1. Sensado
```

```
        double distancia_medida = obtenerDistanciaCM();
```

```
        // 2. Cálculo del Error
```

```
        error_actual = Setpoint - distancia_medida;
```

```
        // 3. Componente Proporcional
```

```
        double proporcional = Kp * error_actual;
```


CONTROL PID APLICADO: CÓDIGO

// 4. Componente Integral (Con protección anti-windup integrada)

```
integral += error_actual * dt;
```

```
// Anti-windup empírico: limita la acumulación para prevenir la saturación extrema del actuador
```

```
integral = constrain(integral, -100, 100);
```

```
double termino_integral = Ki * integral;
```

// 5. Componente Derivativa (Cambio del error en el tiempo)

```
derivativa = (error_actual - error_anterior) / dt;
```

```
double termino_derivativo = Kd * derivativa;
```

// 6. Ecuación General del PID

```
salida_pid = proporcional + termino_integral + termino_derivativo;
```

// Almacenar el error para la siguiente iteración

```
error_anterior = error_actual;
```

// 7. Mapeo y Saturación de la salida hacia el actuador (PWM de Arduino de 0 a 255)

```
// El signo de la salida dependerá de cómo se configure el sentido físico del ventilador.
```

```
int pwm_motor = constrain(abs(salida_pid), 0, 255);
```


CONTROL PID APLICADO: CÓDIGO

// Si el error es despreciable (Zona muerta), detenemos el motor para evitar vibraciones infinitas

```
if (abs(error_actual) < 0.5) {  
    pwm_motor = 0;  
    integral = 0; // Limpiamos la memoria acumulada al llegar a la meta  
}
```

// 8. Actuación de Potencia

```
analogWrite(PIN_ENA, pwm_motor);
```

// 9. Telemetría para el Serial Plotter / Monitor de Arduino

```
Serial.print("Setpoint:"); Serial.print(Setpoint); Serial.print(",");  
Serial.print("Distancia:"); Serial.print(distancia_medida); Serial.print(",");  
Serial.print("Error:"); Serial.print(error_actual); Serial.print(",");  
Serial.print("PWM:"); Serial.println(pwm_motor);  
}
```

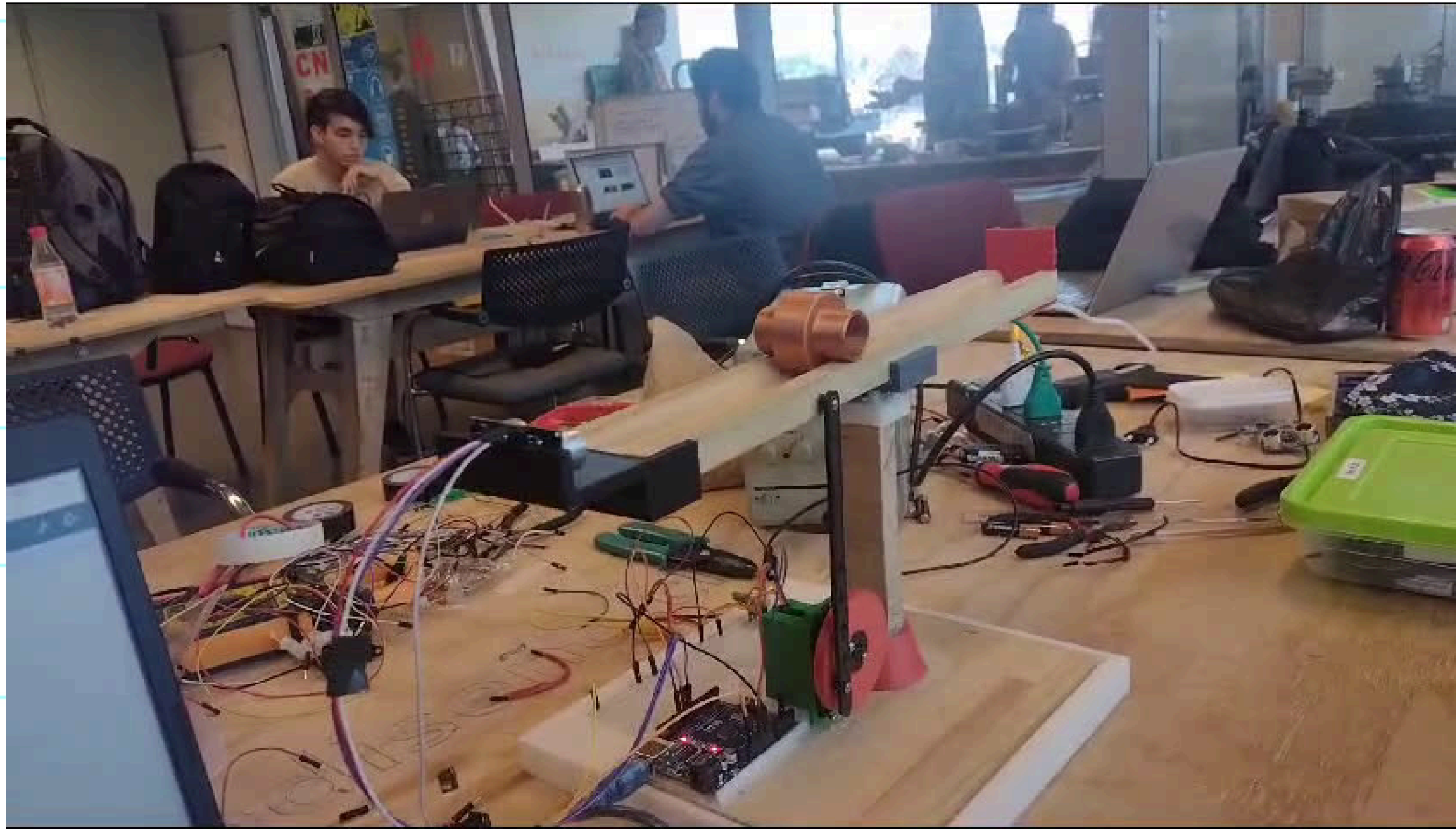
```
}
```

LEVITATING PING-PONG BALL



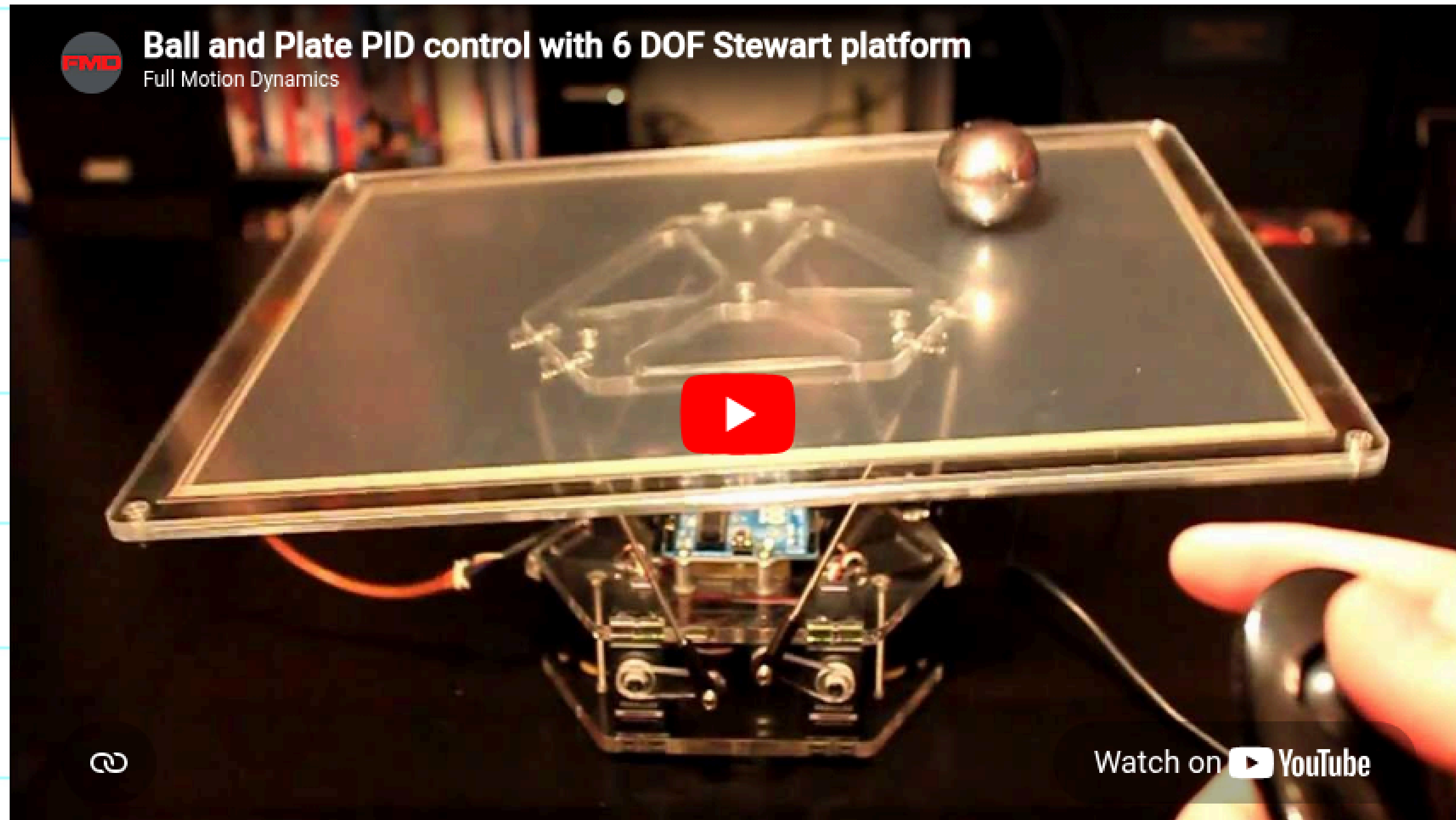
[LINK VIDEO](#)

BALL AND BEAM

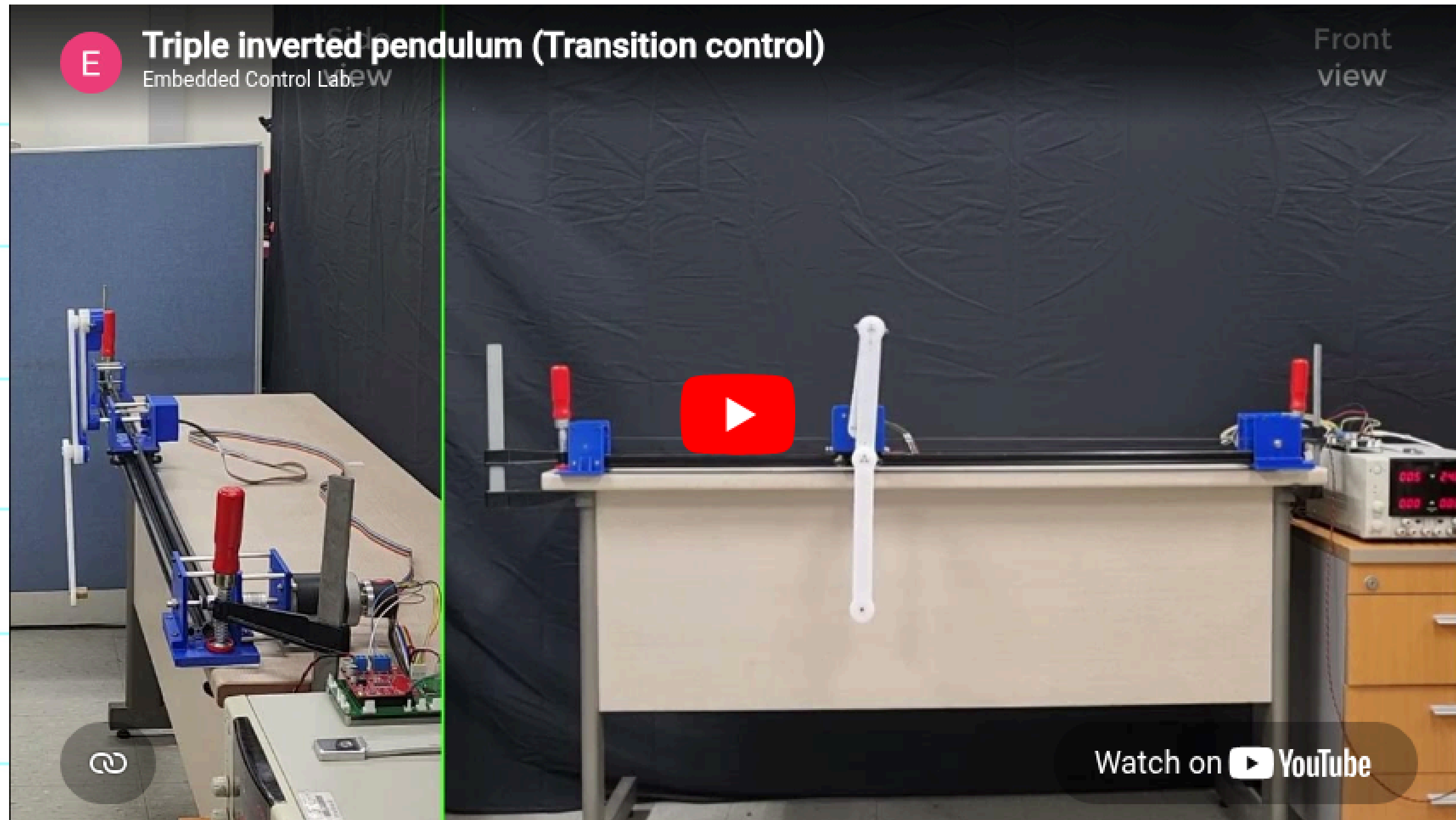


Referencia proyecto

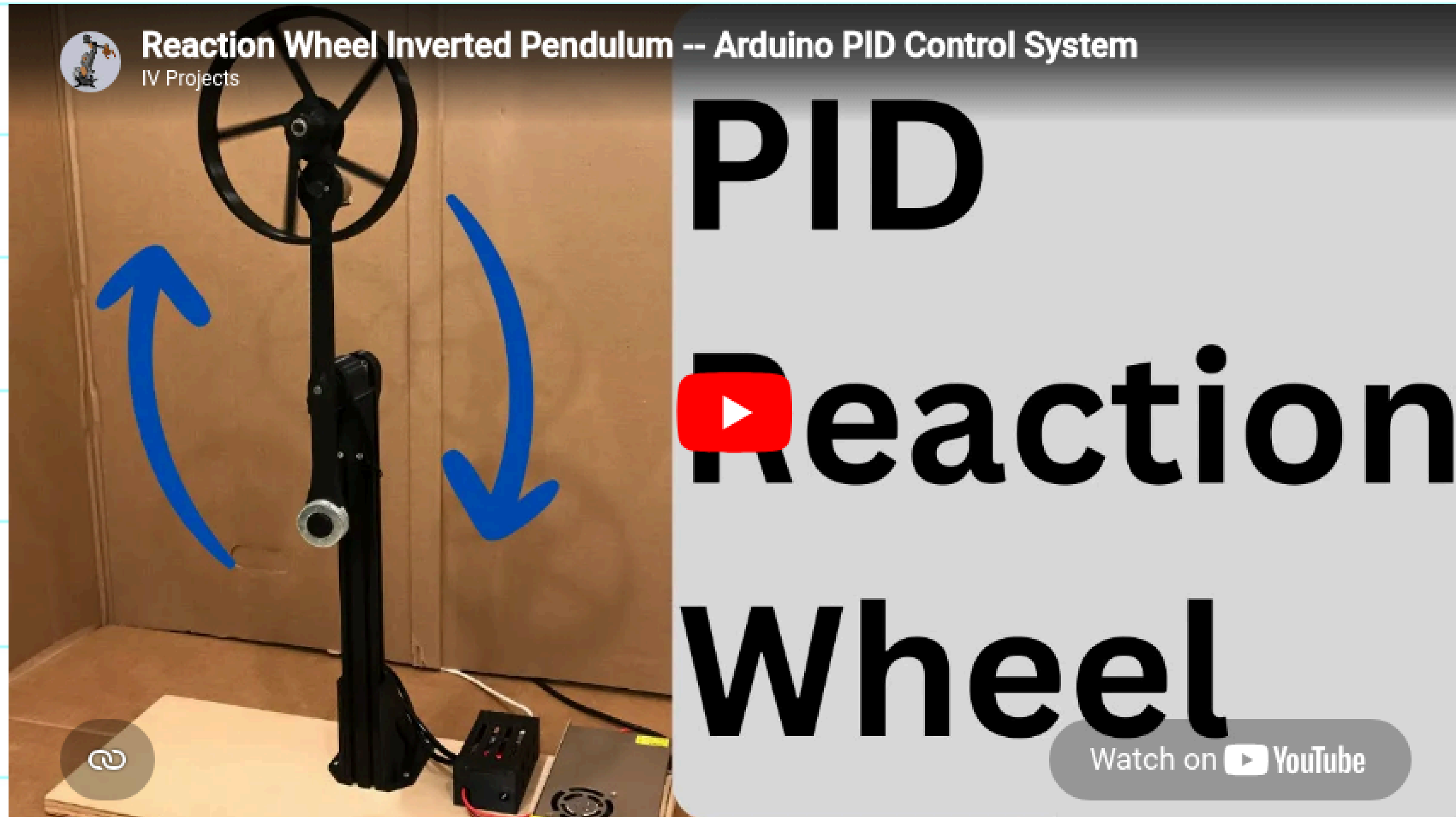
BALL ON PLATE



TRIPLE INVERTED PENDULUM



REACTION WHEEL INVERTED PENDULUM



GRACIAS

